Projeto

Bald Teacher (Professor Careca)

Disciplina: Programação Orientada a Objetos
Tipo: Jogo Digital 2D – Visão Top-Down

Integrantes

Arthur Carvalho, Cauã Aono, Lucas Souza , Nicollas Mota

1. Introdução

1.1 Contextualização do Projeto

O desenvolvimento de jogos digitais é uma área que integra criatividade, lógica e organização de código. Dentro do contexto acadêmico da disciplina de **Programação Orientada a Objetos (POO)**, o projeto **Bald Teacher (Professor Careca)** foi desenvolvido com o objetivo de aplicar, de forma prática, os principais conceitos estudados ao longo do curso.

O jogo foi desenvolvido utilizando tecnologias web e um framework específico para jogos 2D, permitindo trabalhar não apenas a lógica do jogo, mas também a estruturação correta do código, a modularização e a reutilização de classes.

1.2 Objetivo do Jogo

O objetivo do jogo é controlar o personagem **Adriano**, um professor que deve explorar salas de aula dentro de um ambiente universitário, evitando o contato com inimigos (alunos) e coletando itens espalhados pelo cenário. A coleta desses itens representa a recuperação da autoestima do personagem e permite o avanço para novas fases.

1.3 Descrição Geral da Proposta

O jogo possui visão 2D do tipo **top-down**, com foco em exploração, desvio de inimigos e coleta de itens. A proposta foi criar um jogo simples, porém completo, que demonstrasse claramente a aplicação da Programação Orientada a Objetos, o gerenciamento de cenas e a interação entre personagens e ambiente.

2. Tecnologias Utilizadas

2.1 Linguagens e Ferramentas

- **JavaScript (ES6+)**
- **Phaser 3**
- **HTML5**
- **CSS3**
- **Pixel Art**

2.2 Justificativa das Escolhas

O JavaScript foi escolhido por ser uma linguagem amplamente utilizada no desenvolvimento web e por oferecer suporte à Programação Orientada a Objetos. O framework **Phaser 3** foi utilizado por disponibilizar recursos prontos para jogos 2D, como sistema de física, colisões, controle de cenas e áudio, facilitando o desenvolvimento e permitindo foco na lógica do jogo.

3. Conceitos de Programação Orientada a Objetos Aplicados

3.1 Classes e Objetos

O projeto foi estruturado utilizando classes para representar os principais elementos do jogo, como jogador, inimigos e fases. Cada entidade possui responsabilidades bem definidas, o que facilita a organização e manutenção do código.

3.2 Herança

O conceito de herança foi utilizado para reaproveitar comportamentos comuns entre diferentes entidades do jogo. Classes mais específicas herdam características de classes base, reduzindo repetição de código e tornando o sistema mais organizado e escalável.

3.3 Polimorfismo

O polimorfismo é aplicado principalmente no comportamento de diferentes tipos de inimigos e cenas. Apesar de compartilharem uma estrutura base, cada entidade ou fase apresenta comportamentos distintos, respeitando o mesmo conjunto de métodos definidos na arquitetura do jogo.

3.4 Organização do Código

O código foi organizado de forma modular, com separação por arquivos e responsabilidades. Foram adotadas boas práticas como nomes claros, organização por pastas e responsabilidade única por classe, garantindo legibilidade e facilidade de manutenção.

4 Estrutura do Projeto

O projeto foi organizado em pastas para separar arquivos de lógica e recursos visuais/sonoros, facilitando a navegação e o entendimento do sistema.

- **assets:** imagens e áudios

- **src:** arquivos de código
- **scenes:** cenas do jogo
- **entities:** classes de personagens

Essa estrutura contribui para um desenvolvimento mais organizado e profissional.

5. Funcionalidades do Jogo

5.1 Mecânicas Principais

- Movimentação do personagem em várias direções
- Sistema de colisão com paredes e obstáculos
- Inimigos com movimentação autônoma
- Coleta de itens espalhados pelo mapa
- Sistema de vidas representado por corações

5.2 Interação com o Usuário

O jogador interage com o jogo através do teclado, controlando a movimentação do personagem. A interface apresenta informações essenciais, como vida e progresso, de forma simples e visual.

5.3 Regras do Jogo

- O contato com inimigos reduz a vida do jogador
- Ao perder todas as vidas, o jogo é encerrado
- A coleta completa dos itens permite avançar de fase

6. Uso de Inteligência Artificial no Desenvolvimento

Durante o desenvolvimento do jogo **Bald Teacher**, surgiram dúvidas técnicas relacionadas ao uso do framework Phaser 3, especialmente no ajuste das colisões entre os personagens. Um dos problemas identificados foi que a **área de colisão dos inimigos estava maior do que o esperado**, fazendo com que o jogador fosse atingido mesmo sem contato visual direto.

Para analisar esse problema, foi utilizado o recurso de visualização de colisões do próprio Phaser, o que permitiu observar graficamente o comportamento das hitboxes durante a execução do jogo. A partir disso, ficou claro que o problema não estava no sprite, mas na configuração da física aplicada aos inimigos.

Nesse contexto, duas ferramentas de Inteligência Artificial foram utilizadas como apoio: **ChatGPT** e **Gemini Pro**.

Ao utilizar o **ChatGPT**, foi possível obter explicações mais detalhadas sobre o funcionamento do sistema de física do Phaser, principalmente sobre a diferença entre o tamanho visual do personagem e a área real de colisão. Essa abordagem ajudou a compreender a origem do problema e a direcionar melhor os testes no jogo.

Já o **Gemini Pro** apresentou respostas mais diretas e objetivas, confirmando que o ajuste da hitbox era necessário, porém sem aprofundar tanto nos conceitos envolvidos. Essa ferramenta foi útil principalmente para validar rapidamente a ideia de que o problema estava relacionado à configuração da colisão.

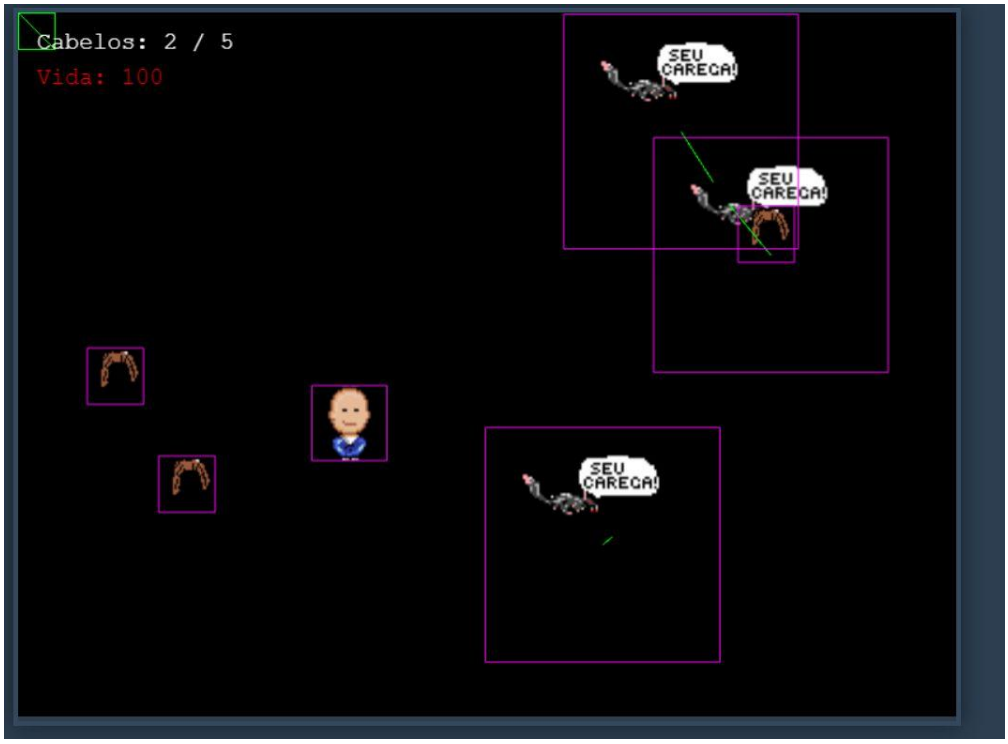
Com base nas análises e testes realizados diretamente no jogo, a área de colisão dos inimigos foi ajustada, resultando em uma jogabilidade mais justa e coerente com o visual apresentado. As ferramentas de IA atuaram apenas como **apoio ao raciocínio e à tomada de decisão**, enquanto toda a implementação e validação foram feitas manualmente pelos desenvolvedores.

7. Registro do Processo de Desenvolvimento

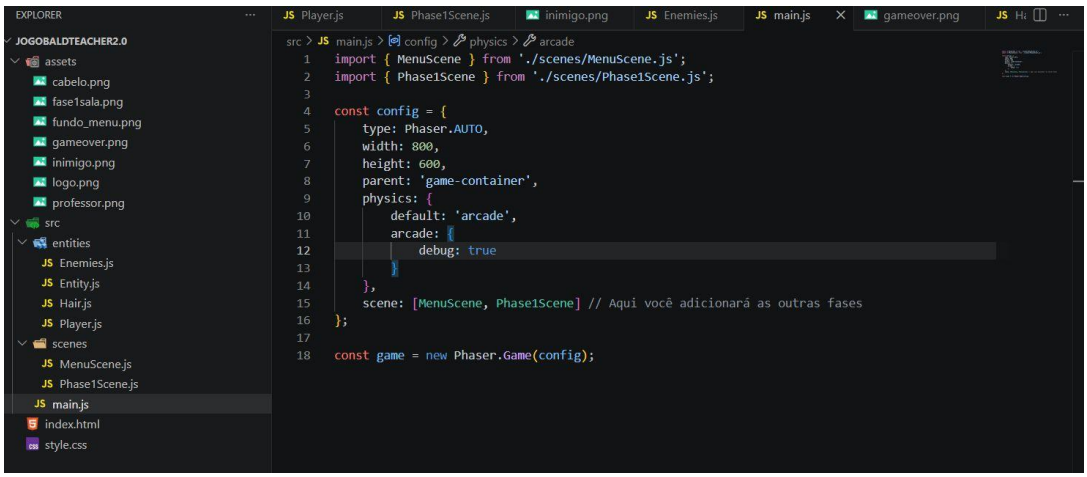
O desenvolvimento do jogo ocorreu de forma gradual, iniciando-se com a criação do protótipo básico e evoluindo com a adição de movimentação, colisões, inimigos, HUD e gerenciamento de cenas. Ao longo do processo, testes constantes foram realizados para garantir o funcionamento adequado de cada funcionalidade.

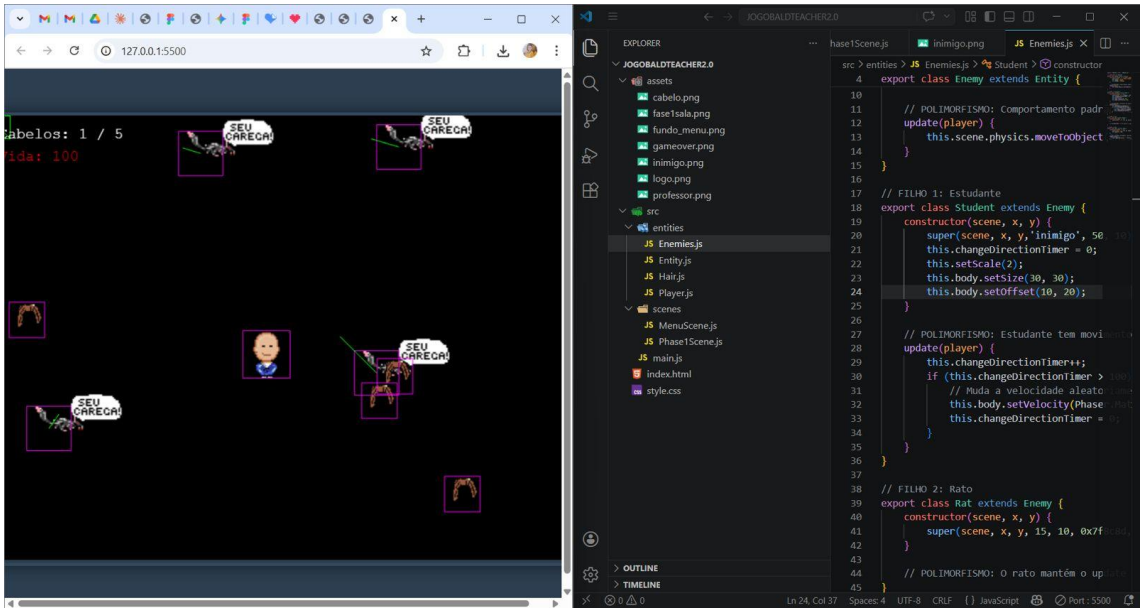
8. Registro de Bugs e Correções

Durante o desenvolvimento, alguns problemas foram identificados, principalmente relacionados às colisões e movimentação dos inimigos. Esses bugs foram analisados por



meio de testes práticos e corrigidos com ajustes na configuração da física do jogo, garantindo maior precisão e melhor experiência para o jogador.





9. Comparativo Entre as IAS

Gemini X Chatgpt

agora quero q me ensine a modificar da minha maneira, como faco para implementar desenhos pixel arte etc

Mostrar raciocínio

Essa é a hora que o jogo ganha vida! Trocar aqueles retângulos coloridos por Pixel Art é o que vai deixar o projeto com cara de jogo profissional.

Como nós estruturamos tudo usando **Programação Orientada a Objetos**, fazer essa mudança vai ser muito mais fácil do que parece. Como a classe `Entity` é a "mãe" de todos os personagens, nós só precisamos ensinar ela a lidar com imagens, e todos os filhos (Player, Inimigos) vão herdar isso automaticamente!

Aqui está o passo a passo para implementar suas Pixel Arts:

Passo 1: Criar a pasta de Assets (Imagens)

1. No seu VS Code, clique numa área vazia do painel lateral esquerdo (Explorer).
2. Crie uma pasta chamada `assets` na **mesma altura** do seu `index.html` e da pasta `src`.
3. Salve suas imagens em Pixel Art dentro dela. **Dica de ouro:** Use sempre o formato `.png` com fundo transparente.

Exemplo de nomes de arquivos: `professor.png`, `aluno.png`, `cabelo.png`.

Passo 2: O método `preload()` nas Cenas

O Phaser precisa carregar as imagens na memória antes do jogo começar. Nós fazemos isso no método `preload`.

Abra o arquivo `src/scenes/Phase1Scene.js` e adicione a função `preload()` logo acima da função `create()`:

JavaScript

agora quero que me ensine a modificar da minha maneira como faço pra importar desenhos pixel arte etc

Boa — agora você entrou na parte mais importante: **customizar o jogo com seus próprios sprites** 🎮🔥
Vou te ensinar do jeito certo, igual quem faz jogo de verdade.

🟢 1. COMO IMPORTAR PIXEL ART PRO JOGO

📁 Passo 1: colocar o arquivo

Coloca sua imagem dentro da pasta:

```
/assets/
```

Exemplo:

```
assets/
├── barata.png
├── professor.png
├── cabelo.png
├── minha_barata_nova.png   👈 (novo sprite)
```

🟢 2. CARREGAR A IMAGEM (preload)

Na sua scene (`Phase2Scene.js` ou `Phase1Scene.js`), no `preload()`:

```javascript
this.load.image('minha_barata', './assets/minha_barata_nova.png');
```

👉 `'minha_barata'` = nome que você usa no código
👉 caminho = onde está o arquivo

---

essa base é do phaser, colisoes? gravidades e bugs?

Mostrar raciocínio

Exatamente, Arthur! Toda essa mágica que montamos na estrutura usa o motor físico padrão do Phaser 3, chamado **Arcade Physics**.

Como o seu trabalho exige que vocês saibam explicar o código na apresentação, aqui está o resumo exato de como as colisões, a gravidade e a prevenção de bugs estão funcionando nessa base que criamos:

1. Colisões (A diferença entre bater e coletar)

Na classe `Phase1Scene.js`, nós usamos dois tipos diferentes de interação física, o que mostra um domínio muito bom da engine para o professor:

- **O Colisor Rígido (`collider`):** Usado entre o Jogador e os Inimigos. Quando eles se tocam, a engine entende que são objetos sólidos e não deixa um passar por dentro do outro.

```javascript
this.physics.add.collider(this.player, this.enemies, this.hitEnemy, null, th
```

- **A Sobreposição (`overlap`):** Usado entre o Jogador e os Cabelos (itens). Se usássemos colisão aqui, o professor iria "bater" no cabelo como se fosse uma parede. Com o `overlap`, o Phaser apenas detecta que um passou por cima do outro, permitindo rodar a função `collectHair` para destruir o item e somar pontos sem parar o movimento.

```javascript
this.physics.add.overlap(this.player, this.hairs, this.collectHair, null, th
```

2. A "False" Gravidade (Atenção aqui para a apresentação!)

essa base é do phaser, colisões? gravidade e bugs?

Sim. Essa base é Phaser, usando principalmente **Arcade Physics**.

O que tem nessa base

Colisões: sim.
Você usa isso:

```javascript
this.physics.add.collider(objeto1, objeto2);
```

Exemplo:

```javascript
this.physics.add.collider(this.player, this.obstacles);
this.physics.add.collider(this.player, this.rats);
```

Overlap/coleta: sim.
Usado para pegar cabelo sem bloquear movimento:

```javascript
this.physics.add.overlap(this.player, this.hairs, this.collectHair, null, this);
```

Gravidade: depende.
Como seu jogo é top-down, tipo mapa visto de cima, normalmente **não usa gravidade**.

No `main.js`, o ideal é:

```javascript
physics: {
```

10. Conclusão

O projeto **Bald Teacher (Professor Careca)** proporcionou uma aplicação prática dos conceitos de Programação Orientada a Objetos, reforçando a importância da organização do código, do uso de classes e da reutilização de comportamentos.

Durante o desenvolvimento, desafios técnicos foram enfrentados, principalmente relacionados à física e colisões, mas esses problemas contribuíram para o aprendizado e aprimoramento das habilidades de análise e resolução de problemas. A Programação Orientada a Objetos mostrou-se fundamental para manter o projeto organizado, escalável e coerente.